ЗВІТ З ЛАБОРАТОРНОЇ РОБОТИ № 1

Розгортання середовища та порівняльний аналіз мовних моделей

Варіант 9

Виконав: студент _____

Група: _____

Перевірив: _____

2026

1. Мета роботи

Ознайомитися з локальним запуском великих мовних моделей, навчитися працювати з Ollama та Python, виконати порівняння швидкості відповідей, якості виконання завдань і впливу інструкцій та параметрів генерації.

2. Використане програмне забезпечення

Операційна система Windows; редактор Visual Studio Code; Python; локальний сервер Ollama; мовні моделі llama3.2:3b та qwen3:4b. Для звернення до Ollama використано HTTP API з бібліотекою urllib.request, а для збереження даних — JSON і CSV.

3. Хід виконання роботи

3.1. Підготовка середовища

Спочатку я встановив Python, Visual Studio Code та Ollama. Потім завантажив моделі llama3.2:3b і qwen3:4b. У робочій папці створив Python-скрипти для запитів до локального API, вимірювання часу та перевірки якості відповідей. Запуск моделей перевірив у терміналі.

3.2. Підключення до Ollama

У файлі providers.py реалізував функцію ask_model(prompt, model). Вона надсилає POST-запит на http://localhost:11434/api/generate, передає назву моделі та запитання і повертає текст відповіді. Такий спосіб дозволив працювати з моделями локально без зовнішнього хмарного API.

3.3. Порівняння часу відповіді

Для обох моделей використав однаковий запит варіанта 9: «Поясни різницю між функцією і методом. Коротко». Кожну модель запустив тричі. Вимірювання виконав за допомогою time.perf_counter(), результати зберіг у results.csv.

Модель	Запуск 1, с	Запуск 2, с	Запуск 3, с	Середній час, с
Llama 3.2 (3B)	11,76	5,68	3,49	6,98
Qwen3 (4B)	102,43	40,55	171,47	104,82

У цьому експерименті Llama 3.2 виявилася приблизно у 15 разів швидшою за Qwen3. При цьому вимірювався повний час отримання відповіді, а не кількість токенів за секунду. На тривалість відповіді Qwen3 міг вплинути режим додаткових міркувань (Thinking).

3.4. Вплив системних інструкцій

Я порівняв два system-повідомлення: «Ти лаконічний технічний асистент» та «Відповідай лише українською, максимум двома реченнями». Для кожної комбінації моделі й інструкції зафіксував довжину відповіді, кількість токенів і час.

Модель	Інструкція	Символів	Токенів загалом	Час, с
Llama 3.2	Лаконічна	181	109	9,20
Llama 3.2	Українська, ≤ 2 речень	225	113	3,14
Qwen3	Лаконічна	121	1313	61,54
Qwen3	Українська, ≤ 2 речень	175	512	19,24

Qwen3 дала точніші визначення функції та методу, тоді як у відповідях Llama 3.2 були змістові й мовні помилки. Попри коротку кінцеву відповідь, Qwen3 використала значно більше токенів, імовірно через внутрішні міркування.

3.5. Перевірка генерації програмного коду

Моделям запропонував написати функцію Python light_travel_time(distance_km), яка повертає час проходження світлом заданої відстані за швидкості 299792.458 км/с. Обидві моделі запропонували правильну формулу: час = відстань / швидкість.

Я окремо перевірив обчислення для 0, 100, 1000 і 300000 км. Для 300000 км правильний результат — приблизно 1,000692286 с. У прикладі Qwen3 цей результат був неточно підписаний як 1,0 с. Llama 3.2 дала працездатну реалізацію, але її пояснення містило мовні помилки.

3.6. Логічне завдання

Задача: у коробці 3 червоні й 2 сині кульки; скільки кульок треба витягнути, щоб гарантовано отримати дві одного кольору? Обидві моделі правильно відповіли: 3 кульки. Llama 3.2 не навела обґрунтування, а Qwen3 пояснила, що після витягування по одній кульці кожного кольору третя обов’язково збігатиметься з одним із них.

3.7. Перевірка української мови

Моделі отримали речення з помилками: «На протязі неділі я приймав участь у міроприємствах по програмуванню». Очікуване виправлення за значення «тиждень»: «Протягом тижня я брав участь у заходах із програмування». Llama 3.2 фактично не виправила речення. Qwen3 спробувала його змінити, але запропонувала помилкові форми «на протягом» і «мероприємствах». Отже, жодна модель не виконала завдання правильно.

3.8. Творче завдання

Моделям запропонував написати історію «Робот, який навчився дружити» українською мовою обсягом 5–7 речень. Llama 3.2 написала 11 речень із численними мовними помилками. Qwen3 написала 5 речень із логічним розвитком сюжету та незначними мовними недоліками. У цьому завданні Qwen3 краще дотрималася вимог.

3.9. Параметри генерації та фактологічні відповіді

Додатково я виконав перевірку фактологічної відповіді у форматі JSON на запитання про перший політ людини в космос. Модель правильно визначила рік 1961 та Юрія Гагаріна; перевірка повернула успішний результат. Для питання про рік випуску першого iPhone використовував температури 0,0; 0,7; 1,0 і додатково top_p = 0,3 за температури 0,7. Під час трьох запусків з top_p = 0,3 отримано однакову відповідь — 2007 рік. Повних числових результатів решти запусків у матеріалах цього звіту немає, тому кількісних висновків щодо різноманітності відповідей не роблю.

4. Підсумкове порівняння

Критерій	Llama 3.2	Qwen3
Генерація коду	4/5	5/5
Логічна задача	4/5	5/5
Українська мова	1/5	2/5
Творче завдання	2/5	4/5
Середня експертна оцінка	2,75/5	4,00/5

Оцінки в таблиці є моєю орієнтовною експертною оцінкою лише за чотирма проведеними завданнями, а не універсальним рейтингом мовних моделей. За якістю пояснень і дотриманням інструкцій Qwen3 загалом показала кращий результат, але обидві моделі помилялися в українській мові.

5. Висновок

Під час лабораторної роботи я навчився встановлювати Ollama, запускати локальні мовні моделі та надсилати до них запити з Python. Я порівняв швидкість Llama 3.2 і Qwen3, перевірив виконання системних інструкцій, генерацію коду, логіку, українську мову й творчі відповіді. У моїх вимірюваннях Llama 3.2 працювала набагато швидше, а Qwen3 частіше давала змістовніші відповіді. Водночас обидві моделі можуть помилятися, тому їхні результати потрібно перевіряти. Отримані дані зберіг у файлах CSV та JSON для подальшого аналізу.

6. Створені файли

Основні файли лабораторної: providers.py, benchmark.py, quality.py, sampling.py, results.csv, quality.json, report.pdf. Додаткові експерименти: hello.py, code_test.py, test_light.py, logic_test.py, ukrainian_test.py, creative_test.py та відповідні JSON-файли результатів. Файл .env не слід публікувати, якщо він містить конфіденційні значення.

7. Пояснення структури дослідження

Розгорнутий додаток містить опис реальних дій у VS Code та Ollama, повні приклади коду, таблиці з отриманими значеннями, окремі графіки, розбір помилок і відповіді на контрольні питання. Частина практичних запитань була додатковою, тому її відділено від обов'язкових пунктів методички.

Основні вихідні дані взято з фактичних результатів запусків, переданих під час роботи. Там, де дані не були зафіксовані, наведено алгоритм вимірювання або позначено «не виміряно». Це дозволяє відтворити дослід, не видаючи припущення за факти.

У методичних вказівках варіант 9 для оцінювання якості передбачає астрономічну тематику: факт про перший політ людини, програмну функцію light_travel_time, задачу про космічний зонд і порівняння зорі з планетою. Для керування випадковістю передбачено назву рекомендаційного сервісу фільмів і питання про перший iPhone.

8. Послідовність налаштування Windows

Роботу почав із перевірки наявності Python. Після встановлення відкрив термінал PowerShell у VS Code. Для контролю версій використовував команди python --version та ollama --version. Папку Lab1int створив на робочому столі у каталозі OneDrive.

Під час першого підключення через бібліотеку openai виникла помилка завантаження DLL компонента jiter через політику контролю програм Windows. Щоб не змінювати налаштування захисту системи, перейшов на стандартну бібліотеку urllib.request і прямий HTTP-запит до локального API Ollama.

Цей підхід зберіг головну мету роботи — програмно викликати локальну модель. Водночас він відрізняється від рекомендованого методичкою OpenAI-сумісного інтерфейсу, тому це потрібно враховувати під час перевірки відповідності вимогам.

```
python --version
ollama --version
ollama pull llama3.2:3b
ollama pull qwen3:4b
ollama list
python providers.py
```

9. Схема взаємодії Python та Ollama

Коли користувач запускає скрипт, Python формує словник із назвою моделі та текстом запиту. Цей словник перетворюється на JSON і передається HTTP POST-запитом на адресу локального сервера Ollama. Сервер виконує генерацію й повертає JSON із текстом відповіді та службовими метриками.

Параметр stream=False означає, що відповідь надходить одним завершеним повідомленням. Це спрощує код, але не дає окремо виміряти час до першого токена. Для коректного TTFT треба використати потоковий режим і фіксувати момент отримання першого фрагмента.

Локальний сервер працює за адресою http://localhost:11434; запит не потребує хмарного ключа. При цьому завантаження моделей під час інсталяції все ж потребує доступу до мережі.

Елемент	**Функція**
VS Code	Редагування й запуск файлів
Python	Підготовка запиту та обробка JSON
Ollama API	Маршрутизація запиту до моделі
Llama / Qwen	Генерація тексту
CSV / JSON	Збереження результатів

10. Лістинг providers.py

У першому допоміжному файлі реалізовано функцію ask_model. Вона приймає два аргументи: запитання і назву моделі. За замовчуванням використовується llama3.2:3b. Функція повертає тільки текстове поле response, а не повний службовий JSON.

У випадку, коли Ollama не запущена або модель не встановлена, запит завершується помилкою. Тому перед запуском програми важливо перевірити стан сервера й список моделей.

```
import json
from urllib.request import Request, urlopen

def ask_model(prompt, model="llama3.2:3b"):
    url = "http://localhost:11434/api/generate"
    data = {"model": model, "prompt": prompt,
            "stream": False}
    request = Request(
        url,
        data=json.dumps(data).encode("utf-8"),
        headers={"Content-Type": "application/json"}
    )
    with urlopen(request, timeout=180) as response:
        result = json.loads(response.read().decode("utf-8"))
    return result["response"]

if __name__ == "__main__":
    question = "Поясни різницю між функцією і методом. Коротко."
    print("Запитання:", question)
    print("Відповідь моделі:")
    print(ask_model(question))
```

11. Лістинг benchmark.py

Для вимірювання часу використано time.perf_counter(), який підходить для вимірювання інтервалів часу. Для кожної моделі зроблено три послідовні запуски однакового запиту. Усі шість вимірювань збережено у CSV.

Середнє значення обчислюється як сума трьох тривалостей, поділена на три. Результат округлюється до двох знаків після коми. Важливо, що перший запуск може містити витрати на завантаження моделі в оперативну пам'ять, тому для точного порівняння слід окремо фіксувати холодний і теплий старт.

```
import time
import csv
from providers import ask_model

prompt = "Поясни різницю між функцією і методом. Коротко."
models = ["llama3.2:3b", "qwen3:4b"]
results = []
for model in models:
    print(f"\nТестування моделі: {model}")
    times = []
    for i in range(3):
        print(f"\nЗапуск {i + 1}")
        start = time.perf_counter()
        answer = ask_model(prompt, model)
        duration = time.perf_counter() - start
        print("Відповідь:", answer)
        print(f"Час: {duration:.2f} секунд")
        times.append(duration)
        results.append([model, i + 1, round(duration, 2)])
    average = sum(times) / len(times)
    print(f"\nСередній час {model}: {average:.2f} секунд")
with open("results.csv", "w", newline="", encoding="utf-8-sig") as file:
    writer = csv.writer(file)
    writer.writerow(["model", "run", "time_seconds"])
    writer.writerows(results)
print("Тестування завершено!")
```

12. Первинні вимірювання часу

Таблиця містить усі значення з файлу results.csv без додаткового коригування. Видно, що час Llama 3.2 зменшувався від першого до третього запуску, а в Qwen3 значення коливалися значно сильніше.

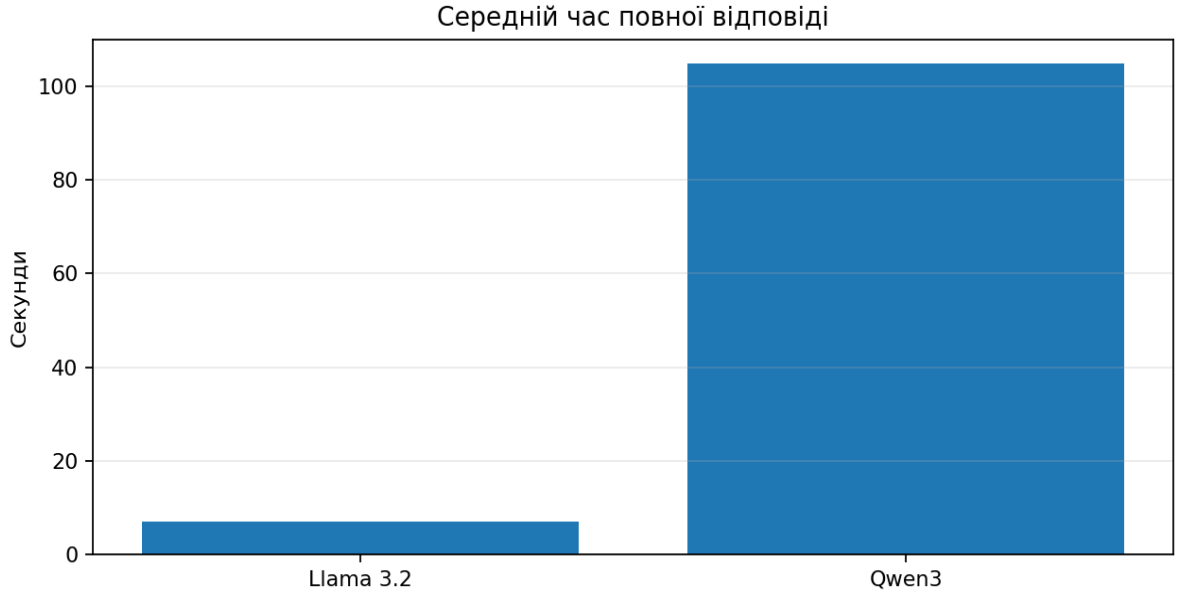
Для Qwen3 перший запуск тривав понад півтори хвилини, другий — близько сорока секунд, третій — майже три хвилини. Ця мінливість показує, що лише середнього значення недостатньо для характеристики комфортності роботи з моделлю.

Модель	№	Час, с
llama3.2:3b	1	11,76
llama3.2:3b	2	5,68
llama3.2:3b	3	3,49
qwen3:4b	1	102,43
qwen3:4b	2	40,55
qwen3:4b	3	171,47

13. Діаграма середнього часу

Середній час Llama 3.2 дорівнює (11,76 + 5,68 + 3,49) / 3 = 6,9767 секунди. Для Qwen3 середній час становить (102,43 + 40,55 + 171,47) / 3 = 104,8167 секунди.

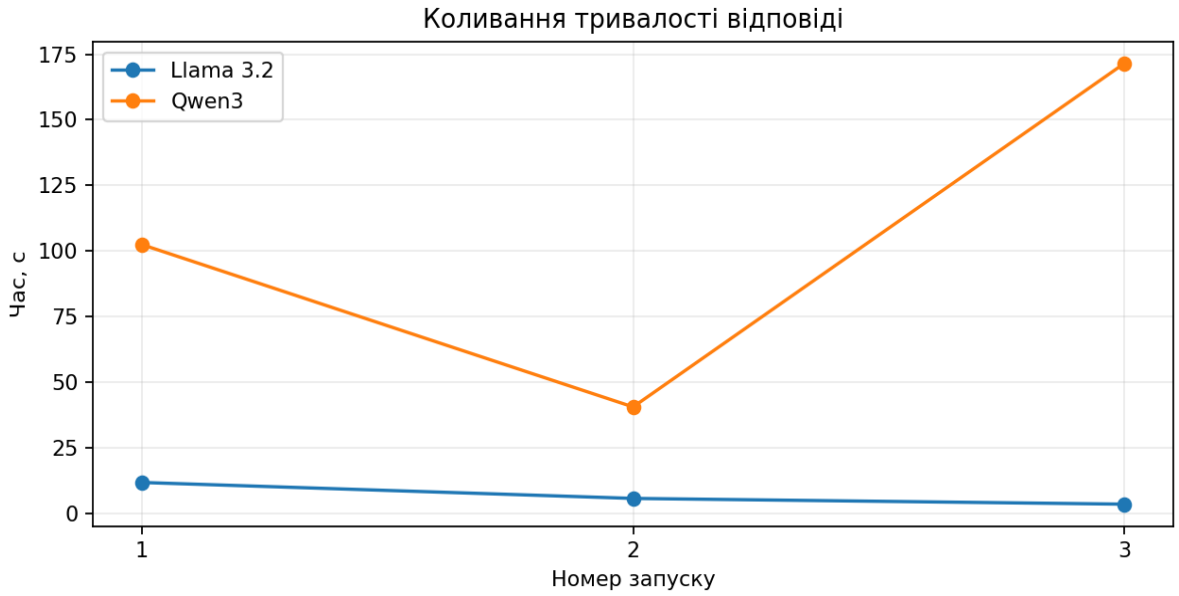
Відношення середніх значень приблизно 15,02. Це означає, що в конкретних умовах експерименту Qwen3 відповідала приблизно в п'ятнадцять разів довше. Узагальнювати цей результат на всі комп'ютери не можна: важливі процесор, відеокарта, оперативна пам'ять, налаштування генерації та довжина відповіді.



14. Графік трьох запусків

Графік дозволяє побачити не тільки різницю середніх значень, а й нестабільність часу відповіді. Для Llama 3.2 усі три вимірювання були меншими за 12 секунд. Для Qwen3 розкид становив від 40,55 до 171,47 секунди.

На практиці це означає, що модель із довгими й непередбачуваними паузами менш зручна для інтерактивного чату. Проте для складних задач, де важливіша правильність, користувач може погодитися чекати довше. Для об'єктивної оцінки слід вимірювати також якість і довжину відповіді.



15. Що саме виміряв benchmark

У створеному скрипті час починається безпосередньо перед викликом ask_model і закінчується після отримання повної відповіді. Це end-to-end latency — повна затримка відповіді. Така метрика включає очікування, обробку запиту, можливе завантаження моделі та генерацію тексту.

TTFT (time to first token) — це інший показник: час до появи першого згенерованого токена. Його не можна відновити з наших шести вимірювань, тому у звіті він позначений як невиміряний. Для повної відповідності методичці потрібен окремий потоковий benchmark.

Крім того, у методичці пропонується перевірити вплив довжини промпту. Ми використовували один короткий промпт, тому залежність часу від довжини запиту також не встановлена.

Показник	**Стан**
Повний час відповіді	Виміряно, 3 рази на модель
TTFT	Не виміряно
Токени/секунду	Не виміряно у benchmark
Короткий / довгий промпт	Не порівнювали
Холодний / теплий старт	Не розділяли формально

16. System-повідомлення: постановка

У цьому досліді я використав API /api/chat замість /api/generate. На відміну від звичайного промпту, API чату дозволяє передати окремі повідомлення з ролями system і user. Системне повідомлення задає бажану поведінку асистента, а повідомлення користувача містить саме запитання.

Перша системна інструкція вимагала лаконічності, друга — української мови й не більше двох речень. Запитання залишалося однаковим. Для кожної пари модель–інструкція зберігався текст відповіді, кількість символів, службові лічильники токенів і повний час.

Позначення	System-текст
S1	Ти лаконічний технічний асистент.
S2	Відповідай лише українською, максимум двома реченнями.

17. Лістинг hello.py, частина 1

Файл hello.py сформував чотири запити до локального чат-API. У запиті параметр temperature встановлено рівним 0,7. Відповідь API зберігалася разом із лічильниками prompt_eval_count та eval_count.

```
import json
import time
from urllib.request import Request, urlopen
PROMPT = "Поясни різницю між функцією і методом. Коротко."
MODELS = ["llama3.2:3b", "qwen3:4b"]
SYSTEMS = [
    "Ти лаконічний технічний асистент.",
    "Відповідай лише українською, максимум двома реченнями."
]
results = []
def ask_model(model, system_message):
    data = {
        "model": model,
        "messages": [
            {"role": "system", "content": system_message},
            {"role": "user", "content": PROMPT}
        ],
        "stream": False,
        "options": {"temperature": 0.7}
    }
    request = Request(
        "http://localhost:11434/api/chat",
        data=json.dumps(data).encode("utf-8"),
        headers={"Content-Type": "application/json"}
    )
    with urlopen(request, timeout=300) as response:
        return json.loads(response.read().decode("utf-8"))
```

18. Лістинг hello.py, частина 2

У другій частині скрипту програма проходить вкладеними циклами по моделях і системних інструкціях, вимірює час та зберігає результат у system_results.json. Поле length містить кількість символів Python-рядка, а не кількість токенів.

```
for model in MODELS:
    for system_message in SYSTEMS:
        print(f"Модель: {model}")
        start = time.perf_counter()
        response = ask_model(model, system_message)
        duration = time.perf_counter() - start
        answer = response["message"]["content"]
        prompt_tokens = response.get("prompt_eval_count", 0)
        completion_tokens = response.get("eval_count", 0)
        results.append({
            "model": model,
            "system": system_message,
            "answer": answer,
            "length": len(answer),
            "prompt_tokens": prompt_tokens,
            "completion_tokens": completion_tokens,
            "total_tokens": prompt_tokens + completion_tokens,
            "time_seconds": round(duration, 2)
        })
with open("system_results.json", "w", encoding="utf-8") as file:
    json.dump(results, file, ensure_ascii=False, indent=4)
```

19. Таблиця відповідей system-тесту

Llama 3.2 з інструкцією S1 дала визначення, яке описувало функцію як залежність виходу від входу, а метод — як послідовність кроків. Для програмування таке пояснення не є точним. З інструкцією S2 відповідь містила мовні домішки й не стала змістовно правильнішою.

Qwen3 з інструкцією S1 пояснила функцію як незалежний блок коду, а метод — як функцію об'єкта. З інструкцією S2 вона сформулювала схоже пояснення українською в двох реченнях. Отже, системна інструкція краще спрацювала в Qwen3.

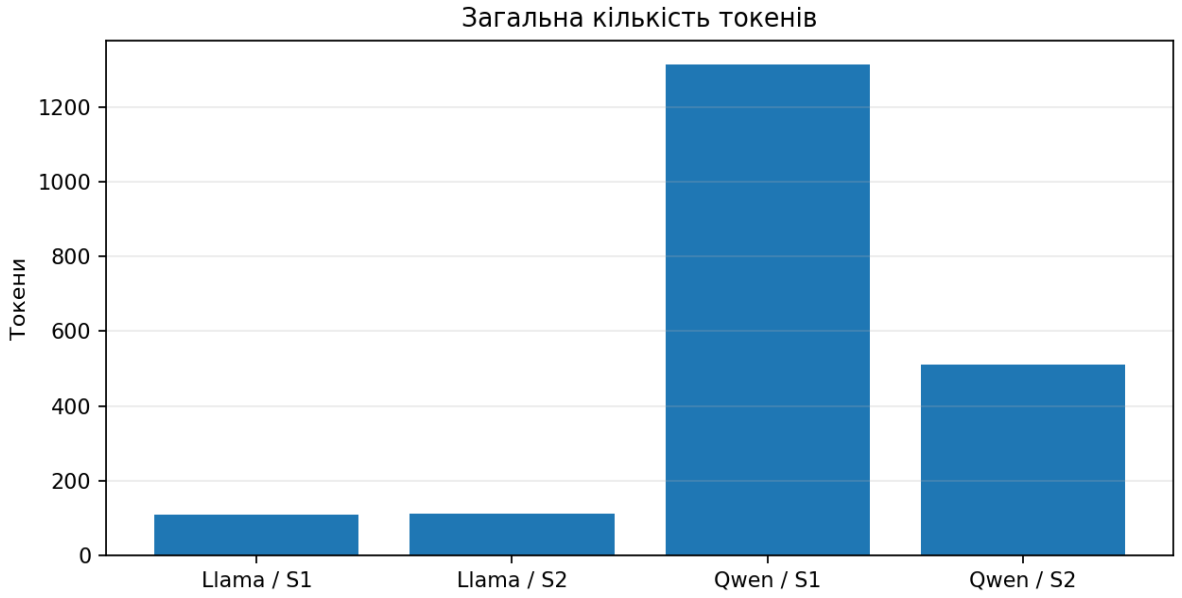
Модель	System	Символів	Час, с
Llama 3.2	S1	181	9,20
Llama 3.2	S2	225	3,14
Qwen3	S1	121	61,54
Qwen3	S2	175	19,24

20. Токени та режим міркування

Службові метрики показали, що Qwen3 використовувала більше токенів генерації, хоча фінальний текст був коротким. Для першої системної інструкції Qwen3 використала 1254 completion-токени, для другої — 444. У Llama 3.2 обидва значення становили 55.

Це узгоджується зі спостереженням, що Qwen3 працювала в режимі додаткових міркувань. Проте лічильники Ollama не дають повного розподілу токенів за призначенням у нашому JSON, тому не можна точно сказати, яка їх частина припала на міркування, а яка — на фінальну відповідь.

Модель / system	Вхід	Генерація	Разом
Llama / S1	54	55	109
Llama / S2	58	55	113
Qwen / S1	59	1254	1313
Qwen / S2	68	444	512



21. Висновки щодо system-повідомлень

Системні інструкції корисні, але не гарантують повної відповідності результату. Вони можуть обмежувати стиль, мову, довжину або формат, проте модель іноді ігнорує окремі вимоги або формально виконує їх, залишаючи помилки у змісті.

Для застосунку, який очікує структуровані дані, однієї текстової інструкції недостатньо. Варто перевіряти JSON парсером, застосовувати схеми, повторювати запит у разі помилки й використовувати приклади потрібного формату. Для української мови додатково потрібна перевірка грамотності.

У нашому експерименті Qwen3 краще виконала змістову частину, але заплатила за це більшим часом і кількістю токенів. Вибір моделі залежить від того, чи пріоритетом є швидкість, чи точність.

22. Генерація коду: умова та формула

Завдання варіанта 9 передбачало функцію light_travel_time(distance_km). На вхід подається відстань у кілометрах, а результатом є час проходження світлом цієї відстані у секундах. У промпті явно задано швидкість 299792.458 км/с.

Формула: t = s / v. Наприклад, для 100 км результат приблизно 0,000333564 секунди; для 1000 км — 0,003335641 секунди. Для 300000 км результат приблизно 1,000692286 секунди.

Обидві моделі написали коректну функцію, яка ділить відстань на константу швидкості світла. Різниця стосувалася переважно оформлення: Llama 3.2 додала довший docstring і приклад, а Qwen3 — компактну реалізацію.

```
def light_travel_time(distance_km):
    speed_of_light = 299792.458
    time_seconds = distance_km / speed_of_light
    return time_seconds

print(light_travel_time(100))
print(light_travel_time(1000))
print(light_travel_time(300000))
```

23. Перевірка light_travel_time

Після отримання коду від моделей я створив окремий файл test_light.py і перевірив функцію на чотирьох значеннях відстані. Така перевірка допомагає виявити помилки, які не завжди помітні при читанні відповіді моделі.

Результат для нульової відстані має бути нульовим. Для додатних відстаней час пропорційно зростає. Застосована формула відповідає вимогам задачі, але не містить перевірки типу вхідних даних чи від'ємних значень; у реальному застосунку ці випадки слід обробляти окремо.

У відповіді Qwen3 приклад із 300000 км був підписаний як 1.0 секунди. Це неточність коментаря: точне значення відрізняється приблизно на 0,000692286 секунди.

Відстань, км	Час, с	Перевірка
0	0,000000000	Правильно
100	0,000333564	Правильно
1000	0,003335641	Правильно
300000	1,000692286	Правильно

24. Додаткова логічна задача з кульками

Під час роботи ми додатково перевірили логічне мислення на задачі про три червоні та дві сині кульки. Запитання: скільки кульок треба витягнути навмання, щоб гарантовано отримати хоча б дві одного кольору? Правильна відповідь — три.

Llama 3.2 відповіла «3 кульки», але не навела пояснення, хоча це було в умові. Qwen3 відповіла так само й пояснила принцип: після двох кульок різних кольорів третя неодмінно збігається за кольором із однією з них.

Цей тест є додатковим і не замінює логічне завдання №5 варіанта 9 з методички. За методичкою потрібно розрахувати час польоту зонда зі швидкістю 60000 км/год на відстань 480 млн км у повних добах.

Модель	Відповідь	Пояснення
Llama 3.2	3	Не навела
Qwen3	3	Навела коректно

25. Обов’язкова логічна задача варіанта 9

Формулювання з методички: «Зонд 60000 км/год, відстань 480 млн км. Скільки повних діб?» Розрахунок виконується у два кроки. Спочатку час у годинах: 480000000 / 60000 = 8000 годин. Потім час у добах: 8000 / 24 = 333,333... доби. Отже, повних діб — 333.

Цей розрахунок наведено як еталон для майбутньої перевірки, але реальні відповіді Llama 3.2 та Qwen3 на цю конкретну умову під час наших запусків не записувалися. Тому оцінки моделей за цим пунктом не виставлено.

Для завершення вимог методички потрібно запустити обидві моделі саме з цим промптом, зберегти їхні відповіді та порівняти не тільки результат, а й послідовність міркувань.

Проміжний показник	**Значення**
Відстань	480 000 000 км
Швидкість	60 000 км/год
Час у годинах	8 000 год
Час у добах	333,333...
Повні доби	333

26. Додаткова перевірка української мови

Моделям було запропоновано виправити речення: «На протязі неділі я приймав участь у міроприємствах по програмуванню». У ньому є лексичні та граматичні помилки, пов'язані з калькуванням і неправильним слововживанням.

Llama 3.2 майже повністю повторила початкове речення та дала неправильне пояснення. Qwen3 замінила частину слів, але написала «на протягом» і «мероприємствах», які також не відповідають нормі.

Коректний варіант за значенням «протягом тижня»: «Протягом тижня я брав участь у заходах із програмування». Ця перевірка показала, що навіть досить переконливі відповіді моделей можуть містити помилки української мови.

Модель	**Результат**
Llama 3.2	Не виправила речення
Qwen3	Виправила неправильно
Еталон	Протягом тижня я брав участь у заходах із програмування

27. Обов’язкове мовне завдання варіанта 9

За методичкою завдання №6 для варіанта 9 звучить: «Чим зоря відрізняється від планети». Це не завдання на редагування помилкового речення. Під час наших додаткових експериментів ми використали інший мовний тест, тому формально пункт методички ще потребує окремого запуску.

Еталонна коротка відповідь: зоря є самосвітним небесним тілом, що випромінює енергію завдяки термоядерним реакціям у надрах; планета зазвичай не виробляє енергію таким способом і переважно відбиває світло зорі, навколо якої обертається.

Для оцінювання моделей потрібно зберегти їхні власні відповіді, перевірити наукову точність, зрозумілість української мови та відповідність обсягу. Еталон не слід видавати за відповідь жодної моделі.

28. Творче завдання про робота

Додатково я попросив моделі написати українською коротку історію на тему «Робот, який навчився дружити» обсягом від п'яти до семи речень. Умовою було також мати початок, розвиток подій і завершення.

Llama 3.2 створила одинадцять речень, перевищила заданий обсяг і припустилася низки мовних помилок. У тексті траплялися змішані написання на зразок «zrozуміти» й іншомовні форми. Qwen3 створила п'ять речень і більш послідовний сюжет про знайомство робота з дитиною.

Цей творчий тест також є додатковим: у методичці для варіанта 9 творче завдання в експерименті з temperature — вигадати назву рекомендаційного сервісу фільмів. Його результати не можна замінити історією про робота.

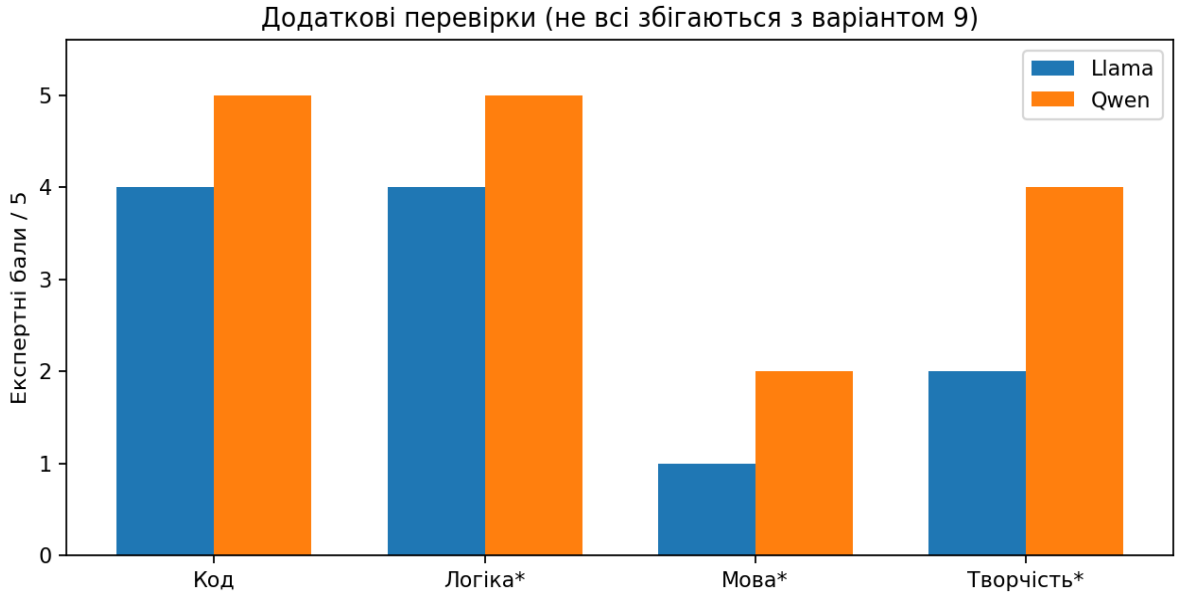
Модель	Речень	Виконання 5–7
Llama 3.2	11	Ні
Qwen3	5	Так

29. Підсумкова експертна рубрика

Орієнтовні оцінки виставлено за чотирма реально проведеними додатковими перевірками. Для коду та логіки обидві моделі продемонстрували правильні основні результати; для української мови обидві помилялися; у творчому завданні Qwen3 краще дотрималася формату.

Оцінки не є результатом стандартизованого тесту, не включають хмарну модель і не замінюють рубрику з методички. Їх варто використовувати лише як опис спостережень у цьому конкретному експерименті.

Критерій	Llama	Qwen
Код	4 / 5	5 / 5
Логіка (кульки)	4 / 5	5 / 5
Редагування мови	1 / 5	2 / 5
Історія про робота	2 / 5	4 / 5
Середнє	2,75	4,00



30. Фактологічна перевірка: космос

За варіантом 9 потрібно перевірити факт про перший політ людини в космос та структуру JSON із полями event, year, person. Під час роботи було запущено quality.py, який сформував quality.json і повідомив про успішну перевірку. Відомості відповідали факту: 1961 рік, Юрій Гагарін.

Цей тест підтверджує, що принаймні один виконаний запит дав очікуваний факт у потрібному форматі. Проте повний JSON із розбивкою за моделями не був переданий до поточного звіту, тому не можна достовірно виставити окремі бали Llama 3.2 і Qwen3 саме за цей запуск.

Для повного оцінювання за методичкою потрібно також перевірити правильність синтаксису JSON, наявність усіх ключів, відсутність зайвого тексту та повторюваність результату.

Поле	**Очікуване значення**
event	Перший політ людини в космос
year	1961
person	Юрій Гагарін

31. Експеримент з temperature

Параметр temperature впливає на випадковість вибору наступних токенів. За нижчих значень відповіді зазвичай стабільніші, а за вищих можуть бути різноманітнішими, але також менш надійними для фактологічних запитань.

Під час нашої роботи запускалися перевірки року випуску першого iPhone за temperature 0,0; 0,7; 1,0, загалом дев'ять проб. Відповіді збережено у sampling_results.json. Повний вміст цього файлу не було надано для аналізу, тому кількість унікальних відповідей за кожним режимом у звіті не вигадую.

У методичці вказані інші рівні температури для систематичного порівняння: 0,0; 0,7; 1,5, по п'ять запусків. Тому наш експеримент слід розглядати як попередній, а не як повне виконання таблиці методички.

Температура	Кількість запусків	Унікальні відповіді
0,0	3	Не зафіксовано в наданих даних
0,7	3	Не зафіксовано в наданих даних
1,0	3	Не зафіксовано в наданих даних

32. Експеримент з top_p

Параметр top_p визначає, з якої частини розподілу ймовірностей модель обирає наступний токен. Нижче значення звужує множину кандидатів, вище дозволяє враховувати ширший набір токенів.

У додатковому тесті я встановив temperature 0,7 та top_p 0,3 і тричі запитав рік випуску першого iPhone. Усі три відповіді збіглися: 2007 рік. Це свідчить про стабільність саме цих трьох запусків, але не гарантує, що будь-який наступний запуск дасть такий самий результат.

Методичка передбачає top_p 0,1; 0,5; 1,0 та творче завдання з назвою сервісу рекомендацій фільмів. Ці значення й творчу метрику кількості унікальних назв ще потрібно виміряти окремо.

Параметр	**Значення**
temperature	0,7
top_p	0,3
Кількість запусків	3
Правильних відповідей	3 з 3
Унікальних відповідей	1

33. Як правильно завершити sampling за методичкою

Для завершення цього етапу треба підготувати два промпти: творчий — «Придумай назву рекомендаційного сервісу фільмів», фактологічний — «У якому році випустили перший iPhone?». Для творчого промпту варто виконати по п'ять запусків при temperature 0,0; 0,7; 1,5 та окремо при top_p 0,1; 0,5; 1,0.

Потім необхідно по п'ять разів поставити фактологічне питання при temperature 0,0 та 1,5. Для кожного режиму обчислюють кількість правильних відповідей. Нарешті, при temperature 0,0 потрібно зробити десять повторів для перевірки відтворюваності.

Результати цього розширеного досліду поки відсутні. Нижче наведено форму таблиці для майбутнього заповнення, а не вигадані показники.

Параметр	Значення	Унікальних із 5
temperature	0,0	—
temperature	0,7	—
temperature	1,5	—
top_p	0,1	—
top_p	0,5	—
top_p	1,0	—

34. Локальні та хмарні моделі

Методичка окремо передбачає порівняння локальної та хмарної моделі через єдиний інтерфейс. У нашій роботі було реально налаштовано й перевірено лише локальний Ollama. Хмарні виклики та їхні результати не виконувалися, тому числового порівняння з хмарою немає.

Переваги локальної моделі: робота без постійного підключення до інтернету після завантаження ваг, контроль над локальними даними, відсутність плати за кожен API-запит. Обмеження: залежність від потужності власного комп'ютера, потреба в пам'яті й можливе зниження якості порівняно з більшими системами.

Хмарна модель може бути зручнішою, коли потрібні високі обчислювальні ресурси та стабільна інфраструктура, але зазвичай потребує мережі, облікового запису й уваги до правил передачі даних. Для повного порівняння треба виконати однакові промпти на обох типах моделей.

Критерій	**Локальна**	**Хмарна**
Виконані тести	Так	Ні
Робота офлайн	Можлива після завантаження	Зазвичай ні
Дані	На локальному ПК	Передаються сервісу
Залежність від обладнання	Висока	Нижча для користувача

35. Аналіз похибок і обмежень

Перше обмеження — малий обсяг вибірки. Три запуски на модель дозволяють побачити різницю, але не дають надійної статистичної оцінки для всіх сценаріїв. Друге — різні режими генерації: Qwen3 могла витрачати додаткові токени на міркування, що ускладнює пряме порівняння часу.

Третє — відсутність стандартизованої автоматичної перевірки якості. Оцінки 1–5 виставлено вручну, тому вони частково суб'єктивні. Четверте — окремі промпти відрізнялися від формулювань варіанта 9 у методичці; їхні результати слід позначати як додаткові, а не обов'язкові.

П'яте — не зафіксовано конфігурацію комп'ютера: процесор, відеокарту, оперативну пам'ять і точні версії моделей. Для відтворюваності майбутніх експериментів ці відомості потрібно додати.

Обмеження	**Як усунути**
Три повтори	Зробити 10–20 повторів
Thinking впливає на час	Порівняти з think=false
Суб’єктивні бали	Описати чітку рубрику
Немає TTFT	Використати stream=True
Немає конфігурації ПК	Зафіксувати характеристики

36. Практичні відповіді на контрольні питання

Яка модель швидша? У моєму тесті Llama 3.2: середня затримка 6,98 секунди проти 104,82 секунди для Qwen3. Однак цей результат не є загальним рейтингом продуктивності, бо вимірювалися лише три запити на одному комп'ютері.

Чи завжди довша відповідь означає кращу? Ні. Важливі правильність, виконання формату й відсутність вигаданих фактів; коротка точна відповідь може бути кориснішою за довгу помилкову.

Чи гарантує system-повідомлення виконання інструкції? Ні. У наших результатах були мовні помилки навіть після прямої вимоги відповідати українською.

Які параметри обрати для JSON? Зазвичай низьку температуру й додаткову перевірку структури. Для творчих назв допустима вища температура, якщо важлива різноманітність.

Що важливіше для чат-бота — TTFT чи повний час? Для інтерактивного чату важливо швидко показати перші слова, а для пакетної обробки важливіший загальний час виконання та пропускна здатність.

37. Вміст папки лабораторної

Під час виконання було створено базові скрипти providers.py, benchmark.py, quality.py, sampling.py і результати results.csv, quality.json, sampling_results.json, sampling_top_p_03.json. Додатково створено hello.py, code_test.py, test_light.py, logic_test.py, ukrainian_test.py, creative_test.py та відповідні файли JSON.

Для здачі за методичкою потрібні основні файли та report.pdf. Додаткові результати можна додати як підтвердження проведених експериментів. Якщо у папці є файл .env з ключами або іншими секретами, його не можна публікувати у відкритому репозиторії.

Файл звіту містить дані, які реально були повідомлені після запусків. Він не є підтвердженням наявності всіх файлів на комп'ютері: перед здачею слід перевірити їх у VS Code.

Файл	**Призначення**
providers.py	Запити до Ollama
benchmark.py	Вимірювання часу
quality.py	Фактологічна перевірка
sampling.py	Температура і top_p
results.csv	Шість вимірювань
quality.json	Результат фактологічного тесту
report.pdf	Підсумковий звіт

38. Підсумкові висновки за всією роботою

У ході лабораторної роботи я встановив локальне середовище для мовних моделей і навчився надсилати запити до Ollama з Python. Я використав дві моделі, Llama 3.2 (3B) і Qwen3 (4B), зібрав результати у CSV та JSON і порівняв їхню поведінку на різних типах завдань.

Найпомітніша відмінність стосувалася швидкості. Llama 3.2 відповідала значно швидше, тоді як Qwen3 частіше давала змістовніші пояснення. Особливо це було помітно у визначенні функції та методу, у логічній задачі та творчому завданні. Проте обидві моделі припускалися помилок української мови.

Окремо я переконався, що навіть правильний програмний код потрібно тестувати, а відповіді з фактами — перевіряти. Режими temperature та top_p впливають на характер генерації, але для висновків про різноманітність необхідно виконати повну серію повторів за методичкою.

У майбутньому я б використовував швидшу модель для простих коротких запитів, а модель із кращою якістю пояснень — для складніших завдань, якщо додатковий час очікування прийнятний. Головний висновок: вибирати модель потрібно не лише за швидкістю, а за сукупністю швидкості, точності, формату та вимог конкретної задачі.

39. Додаток А. Контрольний список перед здачею

Після створення звіту потрібно переконатися, що таблиці й графіки відповідають фактичним CSV та JSON. Також бажано доповнити відсутні досліди, щоб повністю виконати методичні вимоги: TTFT, довгі промпти, порівняння з хмарою, логіку про зонд, мовне запитання про зорю та планету, творчу назву сервісу фільмів і повну матрицю sampling.

Для звіту бажано зберегти скриншоти запуску команд ollama list, python benchmark.py та результати quality.py і sampling.py. У цьому документі скриншоти не підмінялися штучно створеними зображеннями; графіки побудовані тільки на відомих числових даних.

Після фінальної перевірки всі файли потрібно помістити в одну папку або архів згідно з вимогами викладача.

Перевірка	**Статус**
Локальні моделі	Виконано
Benchmark 3×2	Виконано
System-тест	Виконано
Додаткові тести якості	Виконано
Повний TTFT і хмара	Потрібно доповнити
Точні варіантні промпти	Частково
Повна матриця sampling	Потрібно доповнити

40. Додаток Б. Перелік команд для відтворення

Нижче наведено команди, які можна послідовно виконати в терміналі VS Code з робочої папки. Перед запуском переконайтеся, що Ollama встановлена, а потрібні моделі завантажені.

Результати кожного скрипту зберігаються у відповідних CSV або JSON файлах. Додаткові файли не замінюють обов'язкових скриптів, але дають змогу перевірити окремі експерименти.

```
ollama --version
ollama list
ollama pull llama3.2:3b
ollama pull qwen3:4b
python providers.py
python benchmark.py
python quality.py
python sampling.py
python hello.py
python code_test.py
python test_light.py
python logic_test.py
python ukrainian_test.py
python creative_test.py
```

41. Додаток В. Джерела й методичні примітки

Основним джерелом постановки завдань є надані методичні вказівки до лабораторної роботи №1 «Розгортання середовища та порівняльний аналіз мовних моделей». У цьому звіті розділено обов'язкові завдання варіанта 9 та додаткові тести, які були виконані в процесі роботи.

Джерелом виміряних числових значень є результати запусків Python-скриптів, надані під час виконання роботи: results.csv, system_results.json, code_results.json, logic_results.json, ukrainian_results.json і creative_results.json. Дані, яких не було, не реконструювалися.

Для перевірки факту про перший iPhone доцільно використати офіційний пресреліз Apple від 9 січня 2007 року (Apple Reinvents the Phone with iPhone): https://www.apple.com/newsroom/2007/01/09Apple-Reinvents-the-Phone-with-iPhone/ . Для факту першого польоту людини в космос можна звернутися до NASA: https://www.nasa.gov/ . Ці адреси наведено як джерела для самостійної перевірки, а не як результати виконаних у роботі веб-запитів.